

Web Crawler and Search Engine Project Documentation

April 27, 2025

Short Report

1. Approach to Designing the Crawler

The web crawler was designed with the following considerations:

- **Seed URLs:** The crawler starts with predefined seed URLs (<https://en.wikipedia.org/wiki/Internet> and <https://en.wikipedia.org/wiki/Pharaoh>).
- **Breadth-First Search (BFS):** A queue is used to implement BFS, ensuring that links are visited in the order they are discovered.
- **Visited URLs Tracking:** A `HashSet` is used to track visited URLs to avoid revisiting pages.
- **Content Extraction:** The `Jsoup` library is used to fetch and parse HTML content. The crawler extracts the main body text from each page.
- **Link Filtering:** The crawler only follows links that belong to Wikipedia and avoids links with fragments (`#`) or irrelevant pages.
- **Crawling Limit:** The crawler is restricted to a maximum of 10 pages to ensure efficiency and avoid overloading the server.

2. How the Inverted Index is Built

The inverted index maps terms to the documents in which they appear, along with their term frequencies. The process involves:

1. **Tokenization:** The text from each document is split into words using a regex.
2. **Normalization:** Words are converted to lowercase, and stop words (e.g., "the", "and") are removed.
3. **Stemming:** Words are reduced to their root form using the `Stemmer` class (e.g., "running" → "run").
4. **Term Frequency Calculation:** For each document, the frequency of each term is calculated.

5. **Index Construction:** For each term, a list of `Posting` objects is created. Each `Posting` contains the document ID and the term frequency.

The inverted index is stored as a `HashMap<String, List<Posting>>`, where the key is the term, and the value is the list of postings.

3. TF-IDF and Cosine Similarity

TF-IDF Calculation

- **Term Frequency (TF):** The weight of a term in a document is calculated as:

$$TF_{\text{weight}} = 1 + \log_{10}(\text{term frequency})$$

- **Inverse Document Frequency (IDF):** The importance of a term across all documents is calculated as:

$$IDF = \log_{10} \left(\frac{\text{Total Documents}}{\text{Number of Documents Containing the Term}} \right)$$

- **TF-IDF:** The TF-IDF score for a term in a document is:

$$TF\text{-}IDF = TF_{\text{weight}} \times IDF$$

Cosine Similarity

To rank documents for a query:

1. **Query Vector:** The query is tokenized, normalized, and stemmed. A TF-IDF vector is computed for the query.
2. **Document Vectors:** Precomputed TF-IDF vectors for each document are used.
3. **Cosine Similarity:** The similarity between the query and each document is calculated as:

$$\text{Cosine Similarity} = \frac{\text{Dot Product of Vectors}}{\text{Norm of Query Vector} \times \text{Norm of Document Vector}}$$

4. **Ranking:** Documents are ranked based on their cosine similarity scores.

4. Challenges and Solutions

Challenge	Solution
Handling large text data and ensuring efficient processing.	Used <code>HashMap</code> for fast lookups and limited the number of crawled pages.
Avoiding irrelevant links during crawling.	Filtered links to include only Wikipedia pages and excluded fragments.
Ensuring accurate query results.	Implemented stemming and stop-word removal to improve relevance.

Challenges and Solutions

User Manual

1. Prerequisites

- **Java Development Kit (JDK):** Ensure JDK 8 or higher is installed.
- **Maven:** Ensure Maven is installed for dependency management.
- **Internet Connection:** Required for crawling web pages.

2. How to Run the Program

1. Clone the Repository:

```
git clone https://github.com/your-repo/Web-Crawler_IR.git
cd Web-Crawler_IR
```

2. Compile and Run:

```
mvn compile exec:java -Dexec.mainClass="webcrawlerir.Main"
```

- ### 3. Output:
- The program will crawl the seed URLs, build the inverted index, and display the results.

3. How to Input a Query

- After the program starts, you will see the message:

```
Welcome to the Wikipedia Search Engine
Enter your query (or type 'exit' to quit):
```

- Enter a query (e.g., *pharaohs of Egypt*) and press Enter.
- The program will display the top 10 documents ranked by relevance.

4. Example Queries

- pharaohs of Egypt
- ancient history
- pyramids and tombs

5. Exiting the Program

Type `exit` and press Enter to terminate the program.